

Universidad Veracruzana

Licenciatura en Ingeniería de Ciberseguridad e Infraestructura de Cómputo

Programación en Red

Manejo de Excepciones

Brayan Usciel Seseña Hidalgo

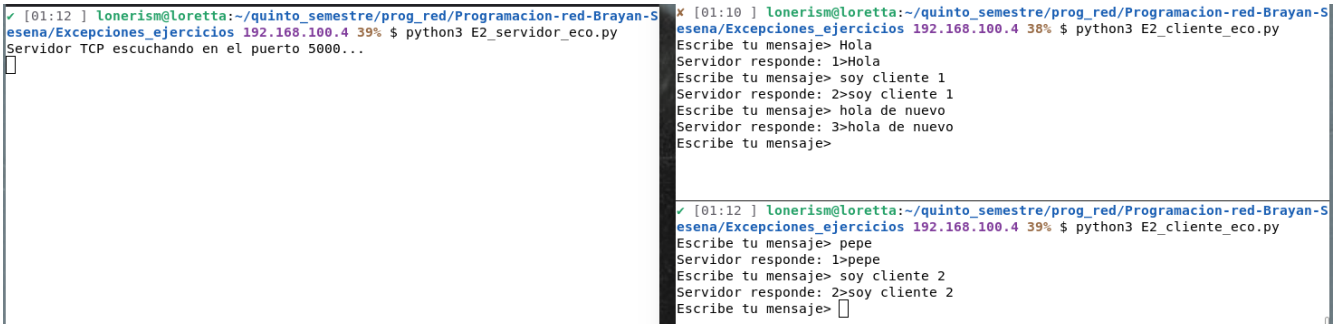
Ejercicios de Consolidación

Ejercicio 1 — Hilos con nombres propios
• Enunciado: a partir del hola mundo con hilos, crear tres hilos cuyos nombres y número de saludos salgan de una lista de tuplas, por ejemplo [("Ana", 3), ("Beto", 5), ("Cora", 2)], y verificar que todos terminan antes del mensaje final.
• **Pistas:** un ciclo for que crea los Thread guardándolos en una lista; un segundo ciclo para hacer join() a cada uno.
• **Se evalúa:** salida entrelazada, mensaje final siempre al final, código limpio.

```
✔ [01:03 ] lonerism@loretta:~/quinto_semestre/prog_red/Programacion-red-Brayan-S
esena/Excepciones_ejercicios 192.168.100.4 29% $ python3 E1_hilos_nombre.py
Ana: hola 0
Beto: hola 0
Cora: hola 0
Ana: hola 1
Beto: hola 1
Cora: hola 1
Ana: hola 2
Beto: hola 2
Beto: hola 3
Beto: hola 4
Fin del programa
✔ [01:03 ] lonerism@loretta:~/quinto_semestre/prog_red/Programacion-red-Brayan-S
```

Por cada persona y su cantidad de saludos asociados, se crea un hilo de ejecución con ayuda de un ciclo for, de igual forma se crea un arreglo que almacena cada hilo para que tras finalizar el target, se llame a la función join() que esperara a que cada hilo termine su ejecución para poder cerrarlo.

Ejercicio 2 — Servidor de eco con contador por cliente
• Enunciado: extender el servidor de eco de la guía anterior para que cada respuesta lleve el número de mensajes que ese cliente ha enviado: el cliente manda hola\n y recibe 1>hola\n, luego 2>otro\n, y así sucesivamente.
• **Pistas:** el contador es una variable local de atender_cliente (un entero que se incrementa por línea recibida); se construye la respuesta con un f-string.
• **Se evalúa:** el conteo es independiente por cliente; dos clientes simultáneos no comparten contador.



El servidor puede atender a varios clientes simultáneamente creando hilos de ejecución para cada uno, a su vez cada cliente tiene su propio contador de mensaje de eco.

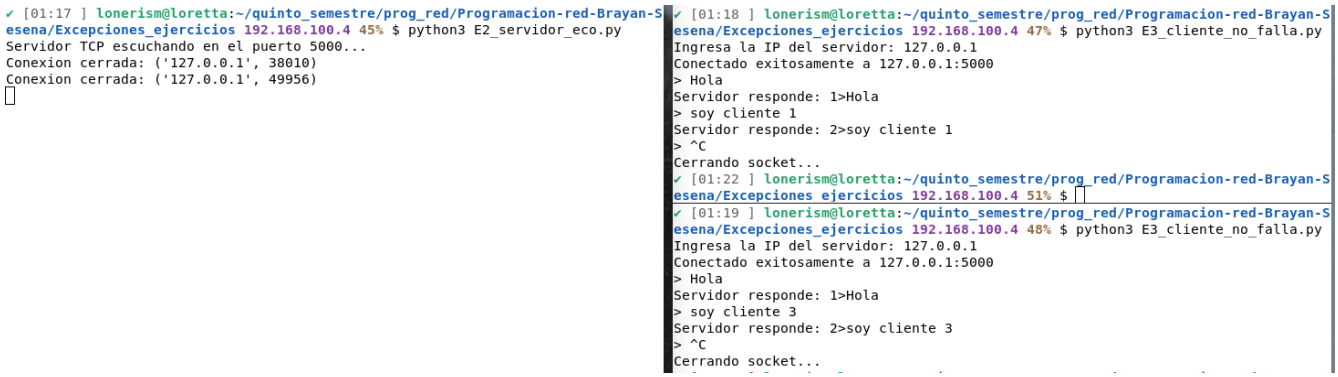
Cada cliente ingresa su mensaje mediante entrada estándar (función input())y se envía al servidor para luego recibirlo en eco.

Ejercicio 3 — Cliente que nunca falla

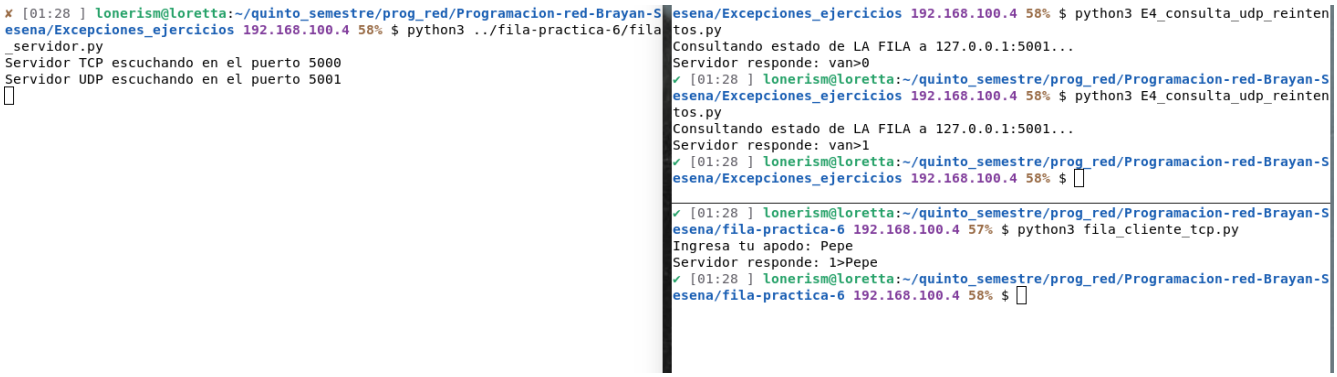
• Enunciado: escribir un cliente TCP que pida al usuario la IP del servidor y maneje todas las salidas de fallo: servidor apagado, IP inválida, respuesta que nunca llega y desconexión a media conversación. En cada caso debe informar con un mensaje claro y terminar con código de salida distinto de cero.

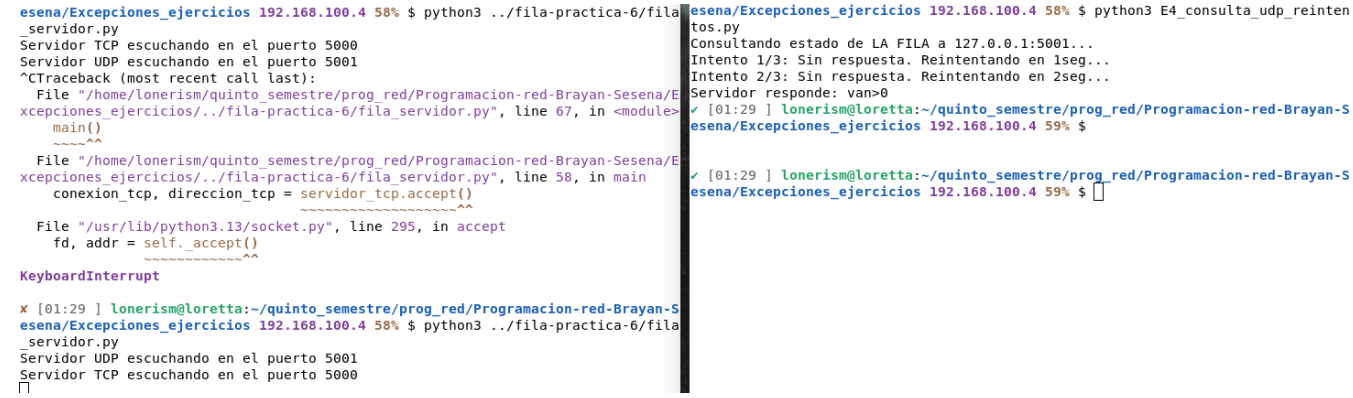
• **Pistas:** Código 3 como base; socket.gaierror aparece cuando la IP o el nombre no se pueden resolver; sys.exit(1) termina con código de error.

• **Se evalúa:** ninguna excepción llega al usuario como traceback; cada fallo tiene su mensaje propio.



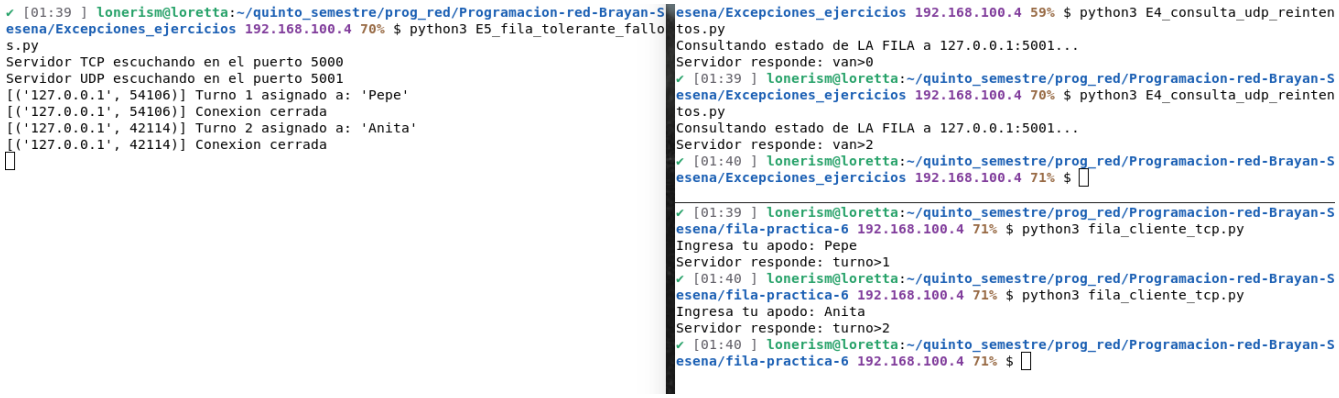
A diferencia del cliente del ejercicio 2, en este cliente se solicita primeramente la dirección IP del servidor para verificar que es posible realizar la conexión, una vez la conexión se establece, se manejan multiples execpciones en caso de errores o cortes de conexión, así como en el caso de que el cliente sea quien el mismo cierre su conexión con la interrupción de teclado (CTRL + C).



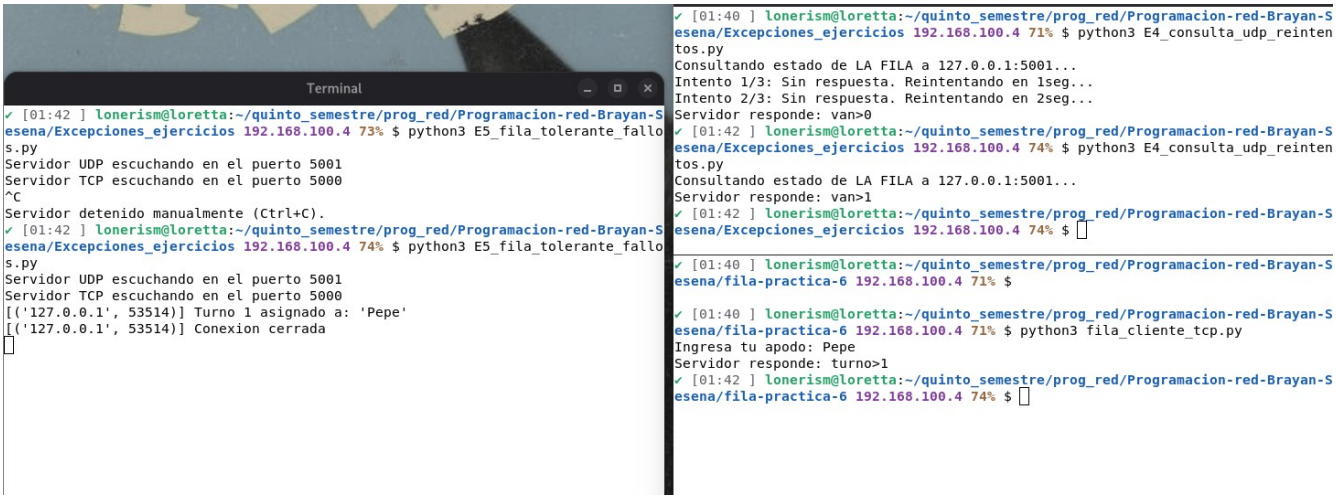


Como complemento se usaron dos programas (servidor la fila y cliente TCP)de una práctica anterior donde se implemento el servidor de LA FILA (razón por la cual tiene tracebacks el servidor).
El cliente UDP envía su consulta normal si el servidor se encuentra encendido, sin embargo, si el servidor esta apagado, realiza sus 3 intentos, si antes de que terminen los 3 intentos el servidor se vuelve a encender, entonces la conexión se restablece.

Ejercicio 5 — LA FILA tolerante a fallos (extensión de la Práctica 6)
• Enunciado: extender el servidor de turnos de la Práctica 6 para que ningún cliente pueda tumbarlo: desconexiones abruptas, mensajes que no siguen el protocolo (algo distinto de TURNO>apodo) y bytes que no son UTF-8 válido deben quedar registrados en consola y el servidor debe seguir asignando turnos sin repetir números.
• **Pistas:** Código 4 como plantilla del hilo de atención; validar el mensaje con startswith("TURNO>") antes de procesarlo; el contador compartido sigue protegido con threading.Lock.
• **Se evalúa:** demostración en clase con un cliente que envía basura a propósito y dos clientes legítimos que reciben turnos consecutivos.



Para LA FILA tolerante a fallos se hizo uso del cliente UDP con reintentos del ejercicio anterior, mientras que el cliente TCP se reutilizo de la practica original de LA FILA.



Esta versión de LA FILA si contiene manejo de excepciones, por lo que no muestra un traceback. Al igual que en el ejercicio anterior, si el servidor esta apagado el cliente UDP inicia sus reintentos, y si en medio de los reintentos el servidor se levanta, la conexión se establece.

Ejercicio 6 — Registro de errores en archivo (desafío)

• Enunciado: agregar al servidor del Ejercicio 5 un registro (log) en un archivo de texto: cada error registrado escribe una línea con fecha, hora, dirección del cliente y descripción, por ejemplo 2026-10-06 10:32:15 (187.141.12.9, 51020) conexión reiniciada por el cliente.

• **Pistas:** time.strftime("%Y-%m-%d %H:%M:%S") da la marca de tiempo; el archivo se abre en modo "a" (agregar) dentro de cada except, o se mantiene abierto protegido con el mismo Lock del contador.

• **Se evalúa:** el archivo crece con cada error provocado en la demostración y ninguna línea queda incompleta aunque dos hilos registren al mismo tiempo.



Para comprobar el registro de los errores en archivo de logs, se modifico al cliente UDP del ejercicio 4, asi en lugar de enviar la consulta con la cadena “CUANTOS” envía la cadena “Añañe” por ejemplo, esto provoca una excepción dado a que el servidor no conoce esa consulta, y entonces se crea el archivo de logs.